WEEK 14 — Unit 5: Continuous Integration (CI) with GitHub Actions

Topic: Caching, Matrix Strategies, Multi-job Workflows, Docker CI, Pushing to Registries & Deployments Course: INT 332 – DevOps

---

Note 1: Caching, Matrix Strategies & Multi-job Workflows

Caching in GitHub Actions

Why Caching Matters

Every time a GitHub Actions workflow runs, it starts on a completely fresh virtual machine. This means every dependency — Maven packages, npm modules, Python packages — must be downloaded from the internet every single time. For a Java Spring Boot project with hundreds of dependencies, this can add several minutes to every build.

Caching solves this by saving downloaded dependencies to GitHub's cache storage after the first run, and restoring them at the start of subsequent runs. If the dependencies have not changed, the restore from cache takes seconds instead of minutes.

How the Cache Action Works

The actions/cache action works in two phases:

- **Restore phase** — at the beginning of the step, it checks if a cache entry matching the key exists and restores it if found
- **Save phase** — at the end of the job (automatically), it saves the specified paths to the cache if the key did not already exist

```
- name: Cache Maven dependencies
  uses: actions/cache@v4
  with:
    path: ~/.m2/repository
    key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
    restore-keys: |
      ${{ runner.os }}-maven-
```

Explanation of fields:

path — the directory to cache. For Maven it is ~/.m2/repository where all downloaded JARs are stored. For npm it is ~/.npm. For pip it is ~/.cache/pip.

key — a unique string that identifies this cache entry. The key includes:

- runner.os — the operating system (Linux, Windows, macOS) so caches are not shared across OS types
- hashFiles('**/pom.xml') — a hash of all pom.xml files in the project. If any pom.xml changes (new dependency added), the hash changes, the key changes, and the cache is invalidated — forcing a fresh download

restore-keys — fallback keys used when an exact key match is not found. Maven will still restore the partial cache from a previous run and only download the newly added dependencies, rather than downloading everything from scratch.

Cache for Different Build Tools

Maven:

```
- uses: actions/cache@v4
  with:
    path: ~/.m2/repository
    key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
    restore-keys: ${{ runner.os }}-maven-
```

npm:

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
    restore-keys: ${{ runner.os }}-npm-
```

Gradle:

```
- uses: actions/cache@v4
  with:
    path: |
      ~/.gradle/caches
      ~/.gradle/wrapper
    key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle*') }}
    restore-keys: ${{ runner.os }}-gradle-
```

Built-in Caching via setup Actions

The actions/setup-java and actions/setup-node actions have a built-in cache parameter that automatically configures caching without needing a separate cache step:

```
- uses: actions/setup-java@v4
  with:
    java-version: '17'
    distribution: 'temurin'
    cache: 'maven'        # Automatically caches ~/.m2/repository
```

This is the simplest approach and is recommended for most Java projects.

Matrix Strategies

What is a Matrix Strategy?

A matrix strategy allows you to run the same job multiple times with different configurations simultaneously. Instead of writing separate jobs for Java 17, Java 21, and Java 11, you define the job once and tell GitHub Actions to run it for each version in parallel.

Basic Matrix Example

```
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        java-version: [11, 17, 21]

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          java-version: ${{ matrix.java-version }}
```

```
      distribution: 'temurin'


  - name: Run tests on Java ${{ matrix.java-version }}
    run: mvn test
```

This creates three parallel jobs — one for Java 11, one for Java 17, one for Java 21 — all running at the same time. The ${{ matrix.java-version }} expression is replaced with the actual value for each job.

Multi-Dimensional Matrix

You can combine multiple variables to test across all combinations:

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest, macos-latest]
    java-version: [17, 21]
```

This creates 6 jobs (3 OS × 2 Java versions), all running in parallel.

Excluding Specific Combinations

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    java-version: [17, 21]
    exclude:
      - os: windows-latest
        java-version: 21
```

This runs all combinations except Windows + Java 21.

Including Additional Variables for Specific Combinations

```
strategy:
  matrix:
    java-version: [17, 21]
    include:
      - java-version: 21
```

```
      experimental: true    # Extra variable for this combination only
```

fail-fast Behaviour

By default, if one matrix job fails, GitHub Actions cancels all other running matrix jobs. To disable this:

```
strategy:
  fail-fast: false
  matrix:
    java-version: [11, 17, 21]
```

With fail-fast: false, all matrix jobs run to completion even if one fails, which is useful when you want full visibility into which versions are failing.

Multi-Job Workflows

A complete CI/CD pipeline typically has multiple jobs that chain together. Each job has a clear responsibility.

Complete Multi-Job Pipeline Example

```
name: Full CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # Job 1: Code quality checks
  lint:
    name: Code Quality Check
    runs-on: ubuntu-latest
    steps:
```

```
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: 'maven'
    - name: Run Checkstyle
      run: mvn checkstyle:check

# Job 2: Build and unit test
build:
  name: Build and Unit Test
  runs-on: ubuntu-latest
  needs: lint
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: 'maven'
    - name: Build and test
      run: mvn clean package
    - name: Upload JAR
      uses: actions/upload-artifact@v4
      with:
        name: app-jar
        path: target/*.jar
```

```
# Job 3: Integration tests
integration-test:
  name: Integration Tests
  runs-on: ubuntu-latest
  needs: build
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: 'maven'
    - name: Run integration tests
      run: mvn verify -DskipUnitTests

# Job 4: Deploy (only on main branch push)
deploy:
  name: Deploy to Staging
  runs-on: ubuntu-latest
  needs: [build, integration-test]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  steps:
    - name: Download JAR
      uses: actions/download-artifact@v4
      with:
        name: app-jar
    - name: Deploy
```

```
      run: echo "Deploying to staging..."
```

Flow:

- lint runs first
- build runs after lint succeeds
- integration-test runs after build succeeds
- deploy runs only after both build and integration-test succeed, and only on a push to main (not on pull requests)
